SAROS

Secure Automotive Real-Time Operating System

Architecture Design Document | Version 1.0

The World's First Automotive-Grade Secure OS

with Native SecOC, Secure Boot, Secure Debug, and Secure Flashing

Author: Thrithin Chand
Senior Automotive Cybersecurity Engineer
March 2026

1. Executive Summary

SAROS (Secure Automotive Real-Time Operating System) is a purpose-built, microkernel-based operating system designed from the ground up for automotive ECUs. Unlike existing solutions that bolt security features onto a general-purpose kernel, SAROS treats security as a first-class architectural citizen — baked into every layer of the OS, from boot to runtime to update.

No open-source or commercial RTOS available today natively integrates SecOC-authenticated IPC, hardware-agnostic crypto abstraction, challenge-response secure debug, and signed firmware flashing within the kernel itself. SAROS fills this gap.

Core Value Proposition

SAROS provides a complete, portable, automotive-grade secure OS where every message is authenticated, every binary is verified, every debug session is authorized, and every firmware update is signed — enforced by the kernel, not middleware.

1.1 Key Differentiators

- Security enforced at kernel level — cannot be bypassed by user space
- Portable Crypto Abstraction Layer (CAL) — runs on NXP CAAM, Renesas SCE, STM STSAFE, ESP32, or pure software mbedTLS
- Full security lifecycle coverage — Boot, Runtime, Debug, Update, Evidence
- Post-quantum crypto ready — algorithm-agile architecture
- ISO 21434 compliant architecture — designed for automotive cybersecurity requirements
- Built by an engineer with 8+ years of production ECU security experience across BMW, Toyota, Ford, and GM programs

2. Problem Statement

Modern vehicles contain 50 to 150 ECUs communicating over CAN, CAN-FD, and Ethernet. Each ECU runs firmware that can be updated, debugged, and configured — creating a large attack surface. Existing OS solutions fail to address this systematically.

Existing Solution	Security Gap	SAROS Answer
FreeRTOS	No secure boot, no message auth, no debug control	All three native in kernel
QNX Neutrino	Security is add-on middleware, not kernel-native	Security is kernel architecture
Zephyr RTOS	Partial secure boot only, no SecOC, no secure debug	Full lifecycle coverage
Linux (automotive)	Too large, not ASIL certified, security bolted on	Minimal microkernel, certifiable
Custom bare metal	No RTOS features, security entirely manual	Full RTOS + security by default

The consequence of these gaps is that automotive OEMs and Tier 1 suppliers spend significant engineering effort manually implementing security features that should be provided by the OS itself — leading to inconsistent implementations, audit failures, and security vulnerabilities.

3. Architecture Overview

SAROS is structured as a layered microkernel architecture. Each layer has a strict boundary — upper layers never bypass lower layers to access hardware directly.

Layer 7 — Application Layer

Vehicle Logic Apps | OTA Agent | Diagnostic Client | Custom Supplier Apps

Apps use only kernel-provided APIs. No direct hardware or crypto access.

Layer 6 — Services Layer

OTA Update Service | UDS Diagnostics (ISO 14229) | Key Management Service

Tamper-Proof Audit Log | VSOC Reporting Agent

Layer 5 — Security Triad (Kernel Services)

Secure Boot | SecOC MAC Engine | Secure Debug | Secure Flashing

These run as privileged kernel services. Cannot be disabled or bypassed.

Layer 4 — Kernel Core

Preemptive Scheduler | Secure IPC (SecOC-authenticated) | Memory Manager (MPU)

Task Manager | Kernel Timer | Kernel Intrusion Detection (IDS)

Layer 3 — Crypto Abstraction Layer (CAL)

Backend: NXP CAAM | Renesas SCE | STM STSAFE | ESP32 HW AES | mbedTLS (SW)

Universal API: aes_cmac | sha256 | ecdsa_sign | ecdsa_verify | rng | key_load

Layer 2 — Hardware Abstraction Layer (HAL)

Timer | UART | CAN-FD | Flash | OTP Fuse | Interrupt Controller

HAL is the ONLY layer that reads/writes hardware registers.

Layer 1 — Hardware

NXP i.MX8QM (CAAM) | Renesas RH850 (SCE) | STM32 (STSAFE) | ESP32 | Generic ARM

OTP Fuses | Flash Storage | CAN Bus | JTAG / Debug Interface

4. Security Lifecycle — Five Phases

SAROS covers every phase of an ECU's security lifecycle. No phase is left unprotected.

Phase	Mechanism	When Active
Power ON	Secure Boot — verified chain of trust	Every boot
Runtime	SecOC MAC — every IPC message authenticated	Always, continuously
Access	Secure Debug — challenge-response authorization	On demand, time-limited
Update	Secure Flashing — signed, verified firmware only	OTA or workshop
Evidence	Tamper-Proof Audit Log — CMAC-signed event record	Always, every event

4.1 Secure Boot

Secure Boot establishes a verified chain of trust from the moment power is applied. Every software component is cryptographically verified before execution. If any verification fails, the ECU halts immediately — no partial boot, no fallback to unsigned code.

Boot Chain

- ROM (immutable, factory-burned) verifies Stage 1 Bootloader
- Stage 1 verifies Stage 2 Bootloader
- Stage 2 verifies SAROS Kernel image
- Kernel verifies each User Space application and driver
- Any failure at any step — system halts, logs event, no recovery without re-flash

Key Features

- ECDSA P-256 signature verification using OEM public key stored in OTP fuse
- SHA-256 hash check of each image before execution
- Anti-rollback protection — firmware version stored in OTP fuse, can only increase
- Panic and halt on any failure — no silent degradation
- All verification performed using HSM — keys never exposed to CPU

4.2 SecOC MAC (Secure Onboard Communication)

SecOC is integrated directly into the SAROS IPC layer. Every message exchanged between tasks or ECUs carries a cryptographic Message Authentication Code (MAC) computed using

AES-128-CMAC. The receiver kernel verifies the MAC before delivering any message to the destination task.

PDU Structure

Field	Size	Purpose
PDU Data (payload)	Up to 64 bytes	Actual message content
Signal ID	1 byte	Identifies the signal type
Freshness Counter	4 bytes (32-bit)	Anti-replay protection
AES-128-CMAC (truncated)	4, 8, or 16 bytes (configurable)	Authentication tag

Key Features

- AES-128-CMAC computed via CAL — hardware-accelerated on supported platforms
- Freshness counter stored in NVM — survives power cycles, prevents replay across reboots
- Configurable MAC truncation — 4 bytes (compact) to 16 bytes (full security)
- Constant-time MAC comparison — prevents timing side-channel attacks
- Replay attack detection — counter must always increase, stale messages dropped and logged
- MAC covers: payload + signal ID + freshness counter — no field can be tampered

4.3 Secure Debug

Debug interfaces (JTAG, UART shell, UDS diagnostic commands) are locked by default on all SAROS builds. Access requires a cryptographic challenge-response authentication using the engineer's private key. Sessions are time-limited and automatically close on timeout or power cycle.

Debug States

- LOCKED — default state on all production builds
- CHALLENGE SENT — ECU has issued a 32-byte random challenge, awaiting response
- AUTHENTICATED — engineer verified, debug access granted for session duration
- PERMANENTLY LOCKED — OTP fuse blown, no debug access ever possible (end-of-life)

Authentication Flow

- Engineer tool sends debug unlock request
- ECU generates 32-byte random challenge using hardware RNG
- Engineer signs challenge with their ECDSA private key

- ECU verifies signature against engineer public key stored in HSM
- On success — session token issued, debug access granted
- On failure — attempt logged, cooldown enforced, repeated failures trigger lockout
- Session auto-closes after configurable timeout (default 5 minutes)

Granular Access Levels

Access Level	Who	Permissions
Level 1 — Read Only	Supplier engineer	Read logs, read ECU state
Level 2 — Diagnostic	OEM engineer	UDS diagnostics, read memory
Level 3 — Full Debug	OEM security team	Full JTAG, memory write, flash
Level 4 — Factory	Production line	Key provisioning, fuse programming

4.4 Secure Flashing

Secure Flashing ensures that only authenticated, signed, and verified firmware can be written to an ECU — whether via physical OBD connection at a workshop or remotely via OTA. Both paths use identical security checks through the Secure Flash Service.

Secure Flash Package (.sfp) Format

Section	Field	Purpose
Header	Magic bytes SRFL, version, ECU Target ID, HW ID	Package identification
Header	Firmware version, minimum version, timestamp	Version and anti-rollback
Crypto	SHA-256 hash of firmware body	Integrity verification
Crypto	ECDSA P-256 signature by OEM key	Authenticity verification
Crypto	Signer ID	Identifies which OEM key was used
Body	Firmware binary (AES-256 encrypted)	Actual firmware payload
Footer	AES-CMAC of entire package	End-to-end integrity check

Flashing Flow — Six Steps

- Step 1 Authentication: challenge-response with OEM/engineer key before any data transfer

- Step 2 Package Verification: check magic, ECU target ID, hardware ID, ECDSA signature, SHA-256 hash
- Step 3 Anti-Rollback: new version must be greater than or equal to OTP fuse version
- Step 4 Write to Inactive Partition: erase partition B, write chunk by chunk with running hash
- Step 5 Post-Write Verify: recompute hash of written data, compare against package hash
- Step 6 Commit and Reboot: update OTP fuse version, signal bootloader, secure boot verifies new image

A/B Partition Strategy

- Partition A — currently running firmware (never touched during update)
- Partition B — receives new firmware during update process
- On successful Secure Boot of B — B becomes active, A becomes backup
- On any failure — ECU automatically boots from A, no manual intervention needed
- Zero brick risk — failed update always recovers to last known good firmware

4.5 Tamper-Proof Audit Log

Every security event in SAROS is recorded to a dedicated audit log partition. Each log entry is signed with AES-CMAC and includes a freshness counter, making entries impossible to forge, delete, or reorder without detection.

- Logged events: every boot (pass/fail), every MAC failure, every debug access, every flash operation, every IDS alert, every key operation
- Each entry: timestamp + event type + details + freshness counter + CMAC signature
- Log partition cannot be erased without OEM-level authentication
- Log integrity verifiable by OEM at any time — supports ISO 21434 audit requirements
- Log entries exportable via secure UDS service to VSOC platform

5. Crypto Abstraction Layer (CAL)

The CAL is the portability backbone of SAROS. It provides a single universal crypto API that SecOC, Secure Boot, Secure Debug, and Secure Flashing all call — never hardware directly. The CAL routes each operation to the correct backend at runtime based on what hardware is available.

5.1 Supported Backends

Platform	Crypto Engine	Key Protection	RNG Quality	Production Grade
NXP i.MX8QM	CAAM	Hardware HSM	True HW RNG	**Yes**
Renesas RH850	SCE / HSM	Wrapped keys	True HW RNG	**Yes**
STM32	STSAFE secure element	Secure element	HW RNG	**Yes**
ESP32	HW AES accelerator	RAM only	HW RNG	Partial
Generic ARM / AVR	mbedTLS (software)	RAM — exposed	SW entropy	**No**

5.2 Universal CAL API

All security components call only these functions. The implementation behind each call is completely transparent to the caller.

- crypto_aes_cmac(key_id, data, len, mac_out) — AES-128-CMAC computation
- crypto_aes_encrypt(key_id, plain, len, cipher_out) — AES encryption
- crypto_sha256(data, len, hash_out) — SHA-256 hash
- crypto_rng(buf, len) — cryptographically secure random bytes
- crypto_ecdsa_sign(key_id, hash, sig_out) — ECDSA P-256 signature
- crypto_ecdsa_verify(key_id, hash, sig) — ECDSA P-256 verification
- crypto_key_load(key_id, key_data, len) — load key into engine
- crypto_key_delete(key_id) — securely erase key from engine

5.3 Engine Selection — Three Methods

Method	Mechanism	Use Case
Compile time	saros.config + Makefile flags	Production build for known hardware
Boot time probe	cal_init() probes hardware registers	Development, multi-platform builds

Runtime switch	crypto switch command in debug shell	Development and testing only

Post-Quantum Readiness

Because all security components call only the CAL API and never hardware crypto directly, SAROS is crypto-agile. When post-quantum algorithms (Kyber for key exchange, Dilithium for signatures) become necessary — only the CAL backend changes. SecOC, Secure Boot, and all other components remain untouched. Vehicles have 10-15 year lifespans. This matters.

6. Kernel Core Features

6.1 Secure IPC

SAROS IPC is the most distinctive kernel feature. Unlike every other RTOS where inter-process messages are unauthenticated, every SAROS IPC message automatically carries SecOC authentication. A task cannot send a raw unauthenticated message — the kernel enforces authentication at the IPC layer.

- Every message: kernel appends freshness counter, computes CMAC via CAL, sends signed PDU
- Every receive: kernel verifies CMAC and freshness before delivering to destination task
- Spoofed or replayed messages are dropped and logged — never reach the application
- Authentication is transparent to the application — no extra code needed

6.2 Trust Level System

Every task and process in SAROS operates at a defined trust level. The kernel enforces communication boundaries between trust levels — a lower-trust process cannot impersonate a higher-trust one.

Trust Level	Who	Access Rights
Level 0 — ROM	Immutable boot code	Absolute, unrestricted
Level 1 — Kernel	SAROS kernel verified by ROM	Full hardware access
Level 2 — Trusted Services	OTA, Key Mgmt, Audit	Kernel APIs, crypto, flash
Level 3 — OEM Apps	Signed by OEM key	Services, IPC, diagnostics
Level 4 — Supplier Apps	Signed by supplier key	Limited IPC, no crypto direct
Level 5 — Sandboxed	Untrusted or test code	Isolated, no system access

6.3 Kernel Intrusion Detection (IDS)

SAROS monitors its own runtime behavior and detects anomalies that indicate attack or compromise. Detection events are logged to the audit log and can trigger escalation to a gateway ECU or VSOC platform.

- Unexpected message patterns on CAN bus — possible injection attack

- Task accessing memory outside its MPU region — possible exploit attempt
- Multiple consecutive MAC failures on same signal — possible spoofing attack
- Debug unlock attempts outside maintenance window — possible physical attack
- Freshness counter jumps or resets — possible replay attack
- Bootloader accessed outside scheduled update window — possible manipulation

7. Competitive Analysis

Feature	FreeRTOS	QNX	Zephyr	SAROS
Secure Boot (native)	No	No	Partial	Yes
SecOC MAC in IPC	No	No	No	Yes
Secure Debug	No	Manual	No	Yes
Secure Flashing	No	No	Partial	Yes
Tamper-Proof Audit Log	No	No	No	Yes
Portable CAL	No	No	Partial	Yes
Kernel IDS	No	No	No	Yes
Trust Level System	No	Partial	No	Yes
A/B Partition OTA	No	No	Partial	Yes
Anti-Rollback	No	No	Partial	Yes
Post-Quantum Ready	No	No	No	Yes
ISO 21434 Native Arch	No	Partial	No	Yes
Hypervisor Path	No	Partial	No	Yes

8. Development Roadmap

Phase	Timeline	Deliverables
Phase 1 — Foundation	Month 1-3	Bare metal boot, UART, basic task scheduler, HAL skeleton
Phase 2 — CAL	Month 4-6	CAL layer with mbedTLS backend, AES-CMAC + SHA-256 working
Phase 3 — Secure Boot	Month 5-7	ECDSA verification, image header format, anti-rollback fuse
Phase 4 — SecOC IPC	Month 7-9	Authenticated IPC, freshness manager, NVM storage
Phase 5 — Secure Debug	Month 8-10	Challenge-response auth, session management, access levels
Phase 6 — Secure Flash	Month 9-11	SFP package format, A/B partition, OTA agent
Phase 7 — CAAM/SCE Port	Month 10-12	NXP i.MX8 and Renesas RH850 CAL backends
Phase 8 — Audit + IDS	Month 12-14	Tamper-proof log, kernel IDS, VSOC reporting
Phase 9 — Release	Month 15-18	Open source release, documentation, consulting launch

First Target Platform

ESP32 with mbedTLS software crypto backend — accessible, affordable hardware with dual-core support and WiFi/BLE for OTA testing. Allows full OS development and validation before porting to NXP i.MX8QM and Renesas RH850.

9. Business Model

Revenue Stream	Model	Target Customer
Open Core License	Kernel open source, security modules commercial	Startups, research
Per-ECU Royalty	License fee per production ECU	Tier 1 suppliers, OEMs
Integration Consulting	Help OEMs integrate SAROS into their platform	OEM engineering teams
Certified Edition	ISO 21434 + ASIL B/D certified build	Safety-critical programs
VSOC Platform	SaaS fleet security monitoring and OTA management	Fleet operators, OEMs
Training	Automotive cybersecurity training on SAROS platform	Tier 1 engineers

10. Author Background

SAROS is designed by an engineer who has implemented every one of its core security primitives in production automotive programs.

Expertise Area	Experience	Relevance to SAROS
Secure Boot / HAB / AHAB	NXP i.MX8QM production ECU	Secure Boot architecture
SecOC + AES-CMAC	BMW, Toyota production CAN-FD	SecOC MAC engine design
HSM Integration	NXP CAAM, Renesas SCE	CAL backend architecture
Key Management	Production key provisioning	Key lifecycle design
ISO 21434 / UNECE WP.29	Full TARA and compliance programs	Security architecture compliance
QNX 7.6 RTOS	Production ECU development	OS kernel design knowledge
Cryptographic Drivers	AES, SHA, ECDSA on hardware	CAL API design
OEM Programs	BMW, Toyota, Ford, GM, Nissan	Real-world requirements

Unique Qualification

No other OS researcher or developer combines QNX production experience, NXP i.MX8 CAAM integration, Renesas RH850 SCE integration, production SecOC implementation, and ISO 21434 compliance work. This combination of skills is the foundation on which SAROS is designed.

11. Next Steps

- Finalize CAL API contract specification — function signatures and error codes
- Define SecOC PDU byte layout and freshness counter format
- Define Secure Flash Package (.sfp) binary format
- Define saros.config configuration file schema
- Set up GitHub repository with layer-by-layer folder structure
- Begin Phase 1 development — bare metal boot on ESP32
- Identify co-founder or early engineering collaborator
- Engage with potential Tier 1 or OEM partners for requirements validation

SAROS — Secure Automotive Real-Time Operating System

Thrithin Chand | thrithin@gmail.com